

管理系统资源

管理系统资源

总体说明

openEuler 虚拟化使用 libvirt 命令来管理虚拟机的系统资源，如 vCPU、虚拟内存资源等。

在开始前：

- 确保主机上运行了 libvirtd 守护进程。
- 用 `virsh list --all` 命令确认虚拟机已经被定义。

管理虚拟 CPU

CPU 份额

概述

虚拟化环境下，同一主机上的多个虚拟机竞争使用物理 CPU。为了防止某些虚拟机占用过多的物理 CPU 资源，影响相同主机上其他虚拟机的性能，需要平衡虚拟机 vCPU 的调度，避免物理 CPU 的过度竞争。

CPU 份额表示一个虚拟机竞争物理 CPU 计算资源的能力大小总和。用户通过调整 `cpu_shares` 值能够设置虚拟机抢占物理 CPU 资源的能力。`cpu_shares` 值无单位，是一个相对值。虚拟机获得的 CPU 计算资源，是与其他虚拟机的 CPU 份额，按相对比例，瓜分物理 CPU 除预留外可用计算资源。通过调整 CPU 份额来保证虚拟机 CPU 计算资源服务质量。

操作步骤

通过修改分配给虚拟机的运行时间的 `cpu_shares` 值，来平衡 vCPU 之间的调度。

- 查看虚拟机的当前 CPU 份额：

```
$ virsh schedinfo
Scheduler   : posix
cpu_shares  : 1024
vcpu_period : 100000
```

```
vcpu_quota   :-1
emulator_period: 100000
emulator_quota :-1
global_period : 100000
global_quota  :-1
iothread_period: 100000
iothread_quota :-1
```

- 在线修改：修改处于 running 状态的虚拟机的当前 CPU 份额，使用带**--live**参数的 virsh schedinfo 命令：

```
$ virsh schedinfo --live cpu_shares=
```

比如将正在运行的虚拟机 openEulerVM 的 CPU 份额从 1024 改为 2048：

```
$ virsh schedinfo openEulerVM --live cpu_shares=2048
Scheduler   : posix
cpu_shares  : 2048
vcpu_period : 100000
vcpu_quota  :-1
emulator_period: 100000
emulator_quota :-1
global_period : 100000
global_quota  :-1
iothread_period: 100000
iothread_quota :-1
```

对 cpu_shares 值的修改立即生效，虚拟机 openEulerVM 能得到的运行时间将是原来的 2 倍。但是这一修改将在虚拟机关机并重新启动后失效。

- 持久化修改：在 libvirt 内部配置中修改虚拟机的 CPU 份额，使用带**--config**参数的 virsh schedinfo 命令：

```
$ virsh schedinfo --config cpu_shares=
```

比如将虚拟机 openEulerVM 的 CPU 份额从 1024 改为 2048：

```
$ virsh schedinfo openEulerVM --config cpu_shares=2048
Scheduler   : posix
cpu_shares  : 2048
vcpu_period : 0
vcpu_quota  : 0
```

```
emulator_period: 0
emulator_quota : 0
global_period : 0
global_quota  : 0
iothread_period: 0
iothread_quota : 0
```

对 cpu_shares 值的修改不会立即生效，在虚拟机 openEulerVM 下一次启动后才生效，并持久生效。虚拟机 openEulerVM 能得到的运行时间将是原来的 2 倍。

绑定 QEMU 进程至物理 CPU

概述

QEMU 主进程绑定特性是将 QEMU 主进程绑定到特定的物理 CPU 范围内，从而保证了运行不同业务的虚拟机不会干扰到邻位虚拟机。例如在一个典型的云计算场景中，一台物理机上会运行多台虚拟机，而每台虚拟机的业务不同，造成了不同程度的资源占用，为了避免存储 IO 密集的虚拟机对邻位虚拟机的干扰，需要将不同虚拟机处理 IO 的存储进程完全隔离，由于 QEMU 主进程是处理前后端的主要服务进程，故需要实现隔离。

操作步骤

通过 virsh emulatorpin 命令可以绑定 QEMU 主进程到物理 CPU。

- 查看 QEMU 进程当前绑定的物理 CPU 范围：

```
$ virsh emulatorpin openEulerVM
emulator: CPU Affinity
-----
*: 0-63
```

这说明虚拟机_openEulerVM_对应的 QEMU 主进程可以在主机的所有物理 CPU 上调度。

- 在线绑定：修改处于 running 状态的虚拟机对应的 QEMU 进程的绑定关系，使用带**--live**参数的 vcpu emulatorpin 命令：

```
$ virsh emulatorpin openEulerVM --live 2-3

$ virsh emulatorpin openEulerVM
emulator: CPU Affinity
-----
*: 2-3
```

以上命令把虚拟机_open__Euler__VM_对应的 QEMU 进程绑定到物理 CPU2、3 上，即限制了 QEMU 进程只在这两个物理 CPU 上调度。这一绑定关系的调整立即生效，但在虚拟机关机并重新启动后失效。

- 持久化绑定：在 libvirt 内部配置中修改虚拟机对应的 QEMU 进程的绑定关系，使用带**--config**参数的 virsh emulatorpin 命令：

```
$ virsh emulatorpin openEulerVM --config 0-3,^1
```

```
$ virsh emulatorpin openEulerVM --config  
emulator: CPU Affinity
```

```
-----
```

```
*: 0,2-3
```

以上命令把虚拟机_open__Euler__VM_对应的 QEMU 进程绑定到物理 CPU0、2、3 上，即限制了 QEMU 进程只在这三个物理 CPU 上调度。**这一绑定关系的调整不会立即生效，在虚拟机下一次启动后才生效，并持久生效。**

调整虚拟 CPU 绑定关系

概述

把虚拟机的 vCPU 绑定在物理 CPU 上，即 vCPU 只在绑定的物理 CPU 上调度，在特定场景下达到提升虚拟机性能的目的。比如在 NUMA 系统中，把 vCPU 绑定在同一个 NUMA 节点上，可以避免 vCPU 跨节点访问内存，避免影响虚拟机运行性能。如果未绑定，默认 vCPU 可在任何物理 CPU 上调度。具体的绑定策略由用户来决定。

操作步骤

通过 virsh vcpupin 命令可以调整 vCPU 和物理 CPU 的绑定关系。

- 查看虚拟机的当前 vCPU 绑定信息：

```
$ virsh vcpupin openEulerVM  
VCPU CPU Affinity
```

```
-----
```

```
0 0-63  
1 0-63  
2 0-63  
3 0-63
```

这说明虚拟机_openEulerVM_的所有 vCPU 可以在主机的所有物理 CPU 上调度。

- 在线调整：修改处于 running 状态的虚拟机的当前 vCPU 绑定关系，使用带**--live**参数的 vcpu vcpupin 命令：

```
$ virsh vcpupin openEulerVM --live 0 2-3
```

```
$ virsh vcpupin openEulerVM  
VCPU CPU Affinity
```

```
-----
```

```
0    2-3  
1    0-63  
2    0-63  
3    0-63
```

以上命令把虚拟机_open__Euler__VM_的 vCPU0 绑定到 PCPU2、3 上，即限制了 vCPU0 只在这两个物理 CPU 上调度。这一绑定关系的调整立即生效，但在虚拟机关机并重新启动后失效。

- 持久化调整：在 libvirt 内部配置中修改虚拟机的 vCPU 绑定关系，使用带**--config**参数的 virsh vcpupin 命令：

```
$ virsh vcpupin openEulerVM --config 0 0-3,^1
```

```
$ virsh vcpupin openEulerVM --config  
VCPU CPU Affinity
```

```
-----
```

```
0    0,2-3  
1    0-63  
2    0-63  
3    0-63
```

以上命令把虚拟机_open__Euler__VM_的 vCPU0 绑定到物理 CPU0、2、3 上，即限制了 vCPU0 只在这三个物理 CPU 上调度。**这一绑定关系的调整不会立即生效，在虚拟机下一次启动后才生效，并持久生效。**

CPU 热插

概述

在线增加（热插）虚拟机 CPU 是指在虚拟机处于运行状态下，为虚拟机热插 CPU 而不影响虚拟机正常运行的方案。当虚拟机内部业务压力不断增大，会出现所有 CPU 均处于较

高负载的情形。为了不影响虚拟机内的正常业务运行，可以使用 CPU 热插功能（在不关闭虚拟机情况下增加虚拟机的 CPU 数目），提升虚拟机的计算能力。

约束限制

- 如果处理器为 AArch64 架构，创建虚拟机时指定的虚拟机芯片组类型(machine)需为 virt-4.1 或 virt 更高版本。如果处理器为 x86_64 架构，创建虚拟机时指定的虚拟机芯片组类型(machine)需为 pc-i440fx-1.5 或 pc 更高版本。
- 在配置 Guest NUMA 的场景中，必须把属于同一个 socket 的 vcpu 配置在同一 vNode 中，否则热插 CPU 后可能导致虚拟机 softlockup，进而可能导致虚拟机 panic。
- 虚拟机在迁移、休眠唤醒、快照过程中均不支持 CPU 热插。
- 虚拟机 CPU 热插是否自动上线取决于虚拟机操作系统自身逻辑，虚拟化层不保证热插 CPU 自动上线。
- CPU 热插同时受限于 Hypervisor 和 GuestOS 支持的最大 CPU 数目。
- 虚拟机启动、关闭、重启过程中可能出现热插 CPU 失效的情况，但再次重启会生效。
- 热插虚拟机 CPU 的时候，如果新增 CPU 数目不是虚拟机 CPU 拓扑配置项中 Cores 的整数倍，可能会导致虚拟机内部看到的 CPU 拓扑是混乱的，建议每次新增的 CPU 数目为 Cores 的整数倍。
- 若需要热插 CPU 在线生效且在虚拟机重启后仍有效，virsh setvcpus 接口中需要同时传入--config 和--live 选项，将热插 CPU 动作持久化。

操作步骤

一、配置虚拟机 XML

1. 使用 CPU 热插功能，需要在创建虚拟机时配置虚拟机当前的 CPU 数目、虚拟机所支持的最大 CPU 数目，以及虚拟机芯片组类型（对于 AArch64 架构，需为 virt-4.1 及以上版本。对于 x86_64 架构，需为 pc-i440fx-1.5 及以上版本）。这里以 AArch64 架构虚拟机为例，配置模板如下：

...

n

hvm

...

说明：

- placement 的值必须是 static。
- m 为虚拟机当前 CPU 数目，即虚拟机启动后默认的 CPU 数目。n 为虚拟机支持热插到的最大 CPU 数目，该值不能超过 Hypervisor 支持的虚拟机最大 CPU 规格及 GuestOS 支持的最大 CPU 规格。n 大于或等于 m。

例如，配一个虚拟机当前 CPU 数目为 4，最大支持的热插 CPU 上限为 64 的 XML 配置为：

```
.....  
    64  
  
    hvm  
  
.....
```

二、热插并上线 CPU

1. 如果热插 CPU 后需要自动上线热插的 CPU，可以使用 root 权限在虚拟机内部创建 udev rules 文件/etc/udev/rules.d/99-hotplug-cpu.rules，并在其中定义 udev 规则，内容参考如下：

```
# automatically online hot-plugged cpu  
ACTION=="add", SUBSYSTEM=="cpu", ATTR{online}="1"
```

说明： 如果没有使用 udev rules 自动上线热插 CPU，可以在热插 CPU 后，使用 root 权限，参考如下命令手动上线：

```
for i in `grep -l 0 /sys/devices/system/cpu/cpu*/online`  
do  
    echo 1 > $i  
done
```

2. 利用 virsh 工具进行虚拟机 CPU 热插操作。例如给虚拟机 openEulerVM 热插 CPU 到 6，且在线生效的参考命令如下：

```
virsh setvcpus openEulerVM 6 --live
```

说明： virsh setvcpus 进行虚拟机 CPU 热插操作的格式如下：

```
virsh setvcpus [--config] [--live]
```

- domain: 参数，必填。指定虚拟机名称。
- count: 参数，必填。指定目标 CPU 数目，即热插后虚拟机 CPU 数目。
- --config: 选项，选填。虚拟机下次启动时仍有效。
- --live: 选项，选填。在线生效。

管理虚拟内存

NUMA 简介

传统的多核运算使用 SMP（Symmetric Multi-Processor）模式：将多个处理器与一个集中的存储器和 I/O 总线相连。所有处理器只能访问同一个物理存储器，因此 SMP 系统也被称为一致存储器访问（UMA）系统。一致性指无论在什么时候，处理器只能为内存的每个数据保持或共享唯一一个数值。很显然，SMP 的缺点是伸缩性有限，因为在存储器和 I/O 接口达到饱和的时候，增加处理器并不能获得更高的性能。

NUMA（Non Uniform Memory Access Architecture）模式是一种分布式存储器访问方式，处理器可以同时访问不同的存储器地址，大幅度提高并行性。NUMA 模式下，处理器被划分成多个“节点”（NODE），每个节点分配一块本地存储器空间。所有节点中的处理器都可以访问全部的物理存储器，但是访问本节点内的存储器所需要的时间，比访问某些远程节点内的存储器所花的时间要少得多。

配置 Host-NUMA

为提升虚拟机性能，在虚拟机启动前，用户可以通过虚拟机 XML 配置文件为虚拟机指定主机的 NUMA 节点，使虚拟机内存分配在指定的 NUMA 节点上。本特性一般与 vCPU 绑定一起使用，从而避免 vCPU 远端访问内存。

操作步骤

- 查看 host 的 NUMA 拓扑结构：

```
$ numactl -H
available: 4 nodes (0-3)
node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
node 0 size: 31571 MB
node 0 free: 17095 MB
node 1 cpus: 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31
node 1 size: 32190 MB
node 1 free: 28057 MB
node 2 cpus: 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47
node 2 size: 32190 MB
node 2 free: 10562 MB
```



```
node 3 cpus: 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63
node 3 size: 32188 MB
node 3 free: 272 MB
node distances:
node 0 1 2 3
0: 10 15 20 20
1: 15 10 20 20
2: 20 20 10 15
3: 20 20 15 10
```

- 在虚拟机 XML 配置文件中添加 numatune 字段，创建并启动虚拟机。例如使用主机上的 NUMA node 0 给虚拟机分配内存，配置参数如下：

假设虚拟机的 vCPU 也绑定在 NODE0 的物理 CPU 上，就可以避免由于 vCPU 访问远端内存带来的性能下降。

说明：

- 分配给虚拟机的内存不要超过该 NUMA 节点剩余的可用内存，否则可能导致虚拟机启动失败。
- 建议虚拟机内存和 vCPU 都绑定在同一 NUMA 节点，避免 vCPU 访问远端内存造成性能下降。例如将上例中 vCPU 也绑定在 NUMA node 0 上。

配置 Guest-NUMA

虚拟机中运行的很多业务软件都针对 NUMA 架构进行了性能优化，尤其是对于大规格虚拟机，这种优化的作用更明显。openEuler 提供了 Guest NUMA 特性，在虚拟机内部呈现出 NUMA 拓扑结构。用户可以通过识别这个结构，对业务软件的性能进行优化，从而保证业务更好的运行。

配置 Guest NUMA 时可以指定 vNode 的内存在 HOST 上的分配位置，实现内存的分块绑定，同时配合 vCPU 绑定，使 vNode 上的 vCPU 和内存在同一个物理 NUMA node 上。

操作步骤

在虚拟机的 XML 配置文件中，配置了 Guest NUMA 后，就可以在虚拟机内部查看 NUMA 拓扑结构。项是 Guest NUMA 的必配项。

[...]

说明：

- 项提供虚拟机内部呈现 NUMA 拓扑功能，“cell id”表示 vNode 编号，“cpus”表示 vCPU 编号，“memory”表示对应 vNode 上的内存大小。
- 如果希望通过 Guest NUMA 提供更好的性能，则需要配置和，使 vCPU 和对应的内存分布在同一个物理 NUMA NODE 上：
- 中的“cellid”和中的“cell id”是对应的；“mode”可以配置为“strict”（严格从指定 node 上申请内存，内存不够则失败）、“preferred”（优先从某一 node 上申请内存，如果不够则从其他 node 上申请）、“interleave”（从指定的 node 上交叉申请内存）；“nodeset”表示指定物理 NUMA NODE。
- 中需要将同一 cell id 中的 vCPU 绑定到与 memnode 相同的物理 NUMA NODE 上。

内存热插

概述

在虚拟化场景下，虚拟机的内存、CPU、外部设备都是软件模拟呈现的，因此可以在虚拟化底层为虚拟机提供内存在线调整的能力。当前 openEuler 版本支持在线给虚拟机添加内存，当虚拟机出现物理内存不足又无法关闭虚拟机的时候，可以使用此特性增加虚拟机的物理内存资源。

约束限制

- 创建虚拟机的时候，AArch64 平台上指定的主板类型(machine)需为 virt-4.1 或更高 virt 以上，x86 平台上指定的主板类型需要为 pc-i440fx-1.5 以上版本。
- 内存热插特性依赖于 Guest NUMA，虚拟机必须配置 Guest NUMA，否则无法完成内存热插流程。
- 热插内存时候必须指定新增内存所属的 Guest NUMA node 编号，否则内存热插失败。
- 虚拟机内核必须支持内存热插能力，否则虚拟机无法识别新增内存或者无法上线内存。
- 配置使用大页的虚拟机，热插内存的容量必须是系统 hugepagesz 的整数倍，否则会导致热插失败。
- 热插内存的大小必须为 Guest 物理内存块大小 block_size_bytes 的整数倍，否则无法正常上线。在 Guest 内部执行 lsmem 可以获取 block_size_bytes 大小。
- 配置 n 个 virtio-net 网卡后，最大可热插次数取值为 $\min\{\text{max_slot}, 64 - n\}$ ，因为要给网卡预留 slot。

- vhost-user 设备和内存热插特性互斥。配置了 vhost-user 设备的虚拟机不支持内存热插；内存热插后，不支持虚拟机热插 vhost-user 设备。
- 如果虚拟机操作系统为 Linux 系列，请确保初始内存大于等于 4GB。
- 如果虚拟机操作系统为 Windows 类型，第一次热插内存必须指定到 Guest NUMA node0 上，否则热插内存无法被虚拟机识别。
- 在直通场景下，由于需要预先分配内存，因此启动和热插内存都比普通虚拟机要慢（尤其是大规格虚拟机），属于正常现象。
- 建议虚拟机可用内存与热插内存的比例至少为 1:32，即热插 32G 内存虚拟机至少需要有 1G 可用内存，如果低于该比例可能会导致虚拟机卡死。
- 热插内存是否自动上线取决于虚拟机操作系统自身逻辑，可以手动上线或者配置 udev 规则自动上线。

操作步骤

一、配置虚拟机 XML

1. 使用内存热插功能，需要在创建虚拟机时配置可热插内存的最大范围、预留槽位号，并配置 Guest NUMA 拓扑结构。

例如，为虚拟机配置 32GiB 初始内存，预留 256 个槽位号，最大支持 1TiB 内存上限，2 个 NUMA node 的配置为：

```
32
1024
....
```

说明： 其中： maxMemory 字段中 slots 号表示预留的内存插槽，最大取值为 256。 maxMemory 表示虚拟机支持的最大物理内存上限。 Guest NUMA 配置请参见“配置 Guest NUMA”相关章节。

二、热插并上线内存

1. 如果热插内存后需要自动上线热插的内存，可以使用 root 权限在虚拟机内部创建 udev rules 文件/etc/udev/rules.d/99-hotplug-memory.rules，并在其中定义 udev 规则，内容参考如下：

```
# automatically online hot-plugged memory
ACTION=="add", SUBSYSTEM=="memory", ATTR{state}="online"
```

2. 根据需要热插的内存大小和虚拟机 Guest NUMA Node 创建内存描述 xml 文件。

例如，热插 1GiB 内存到 NUMA node0 上：

```
1024
0
```

3. 使用 `virsh attach-device` 命令为虚拟机热插内存。其中 `openEulerVM` 为虚拟机名称，`memory.xml` 为热插内存的描述文件，`--live` 表示热插内存在线生效，也可以使用 `--config` 将热插内存持久化到虚拟机 xml 文件中。

```
# virsh attach-device openEulerVM memory.xml --live
```

说明： 如果没有使用 `udev rules` 自动上线热插内存，也可以使用 `root` 权限，参考如下命令手动上线：

```
for i in `grep -l offline /sys/devices/system/memory/memory*/state`
do
    echo online > $i
done
```